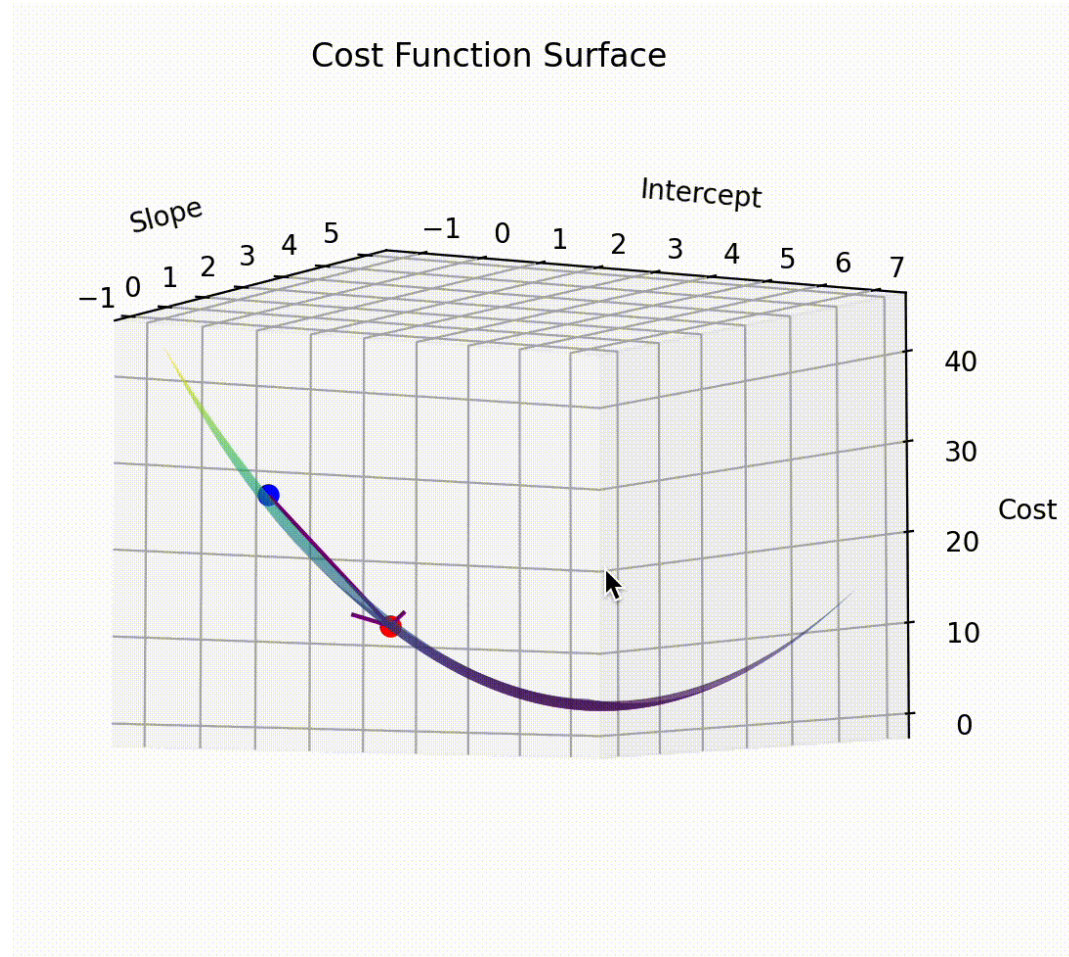


Optimization and Linear Programming

Management Science Principles

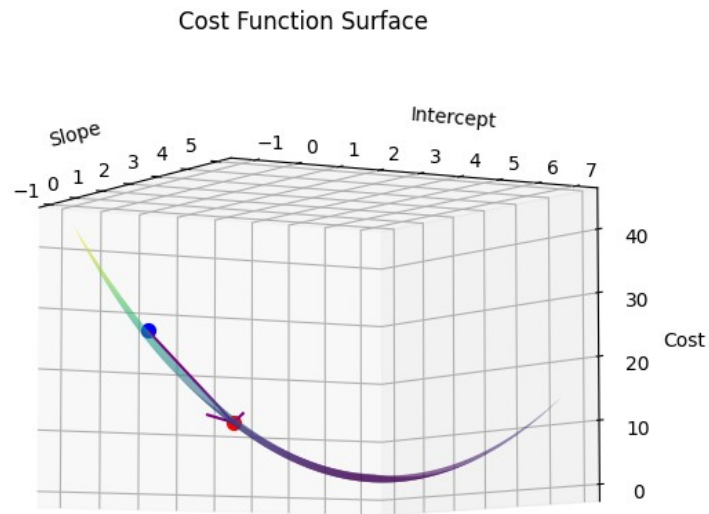
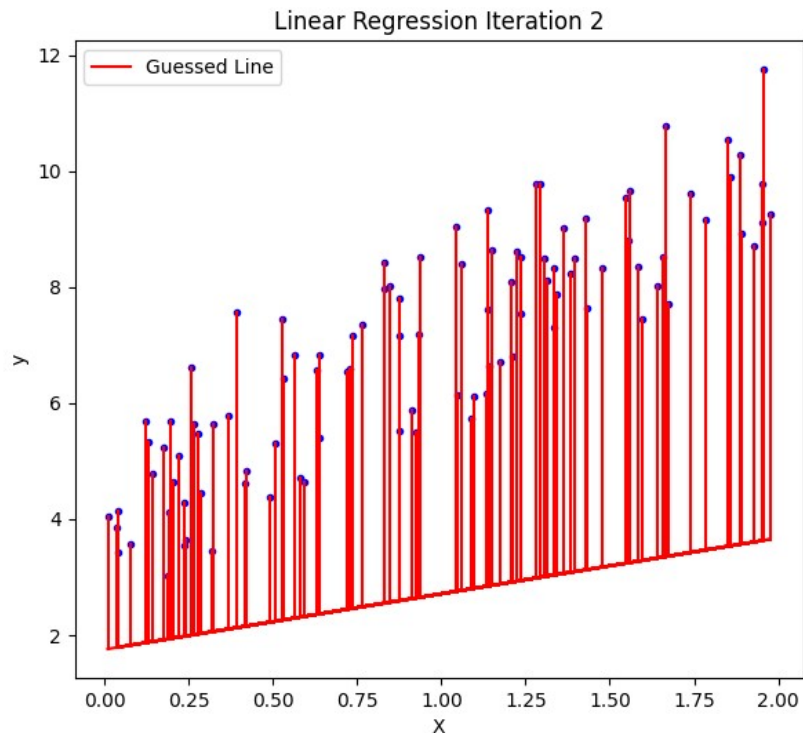
Recall: Gradient Descent

- In 370, we introduced **gradient descent** as an optimization algorithm.
- It is used to update parameter values through the parameter space using the cost or error function.
- Move from points of high error to low error using the directional derivative, or gradient.
 - Hence, *gradient descent*

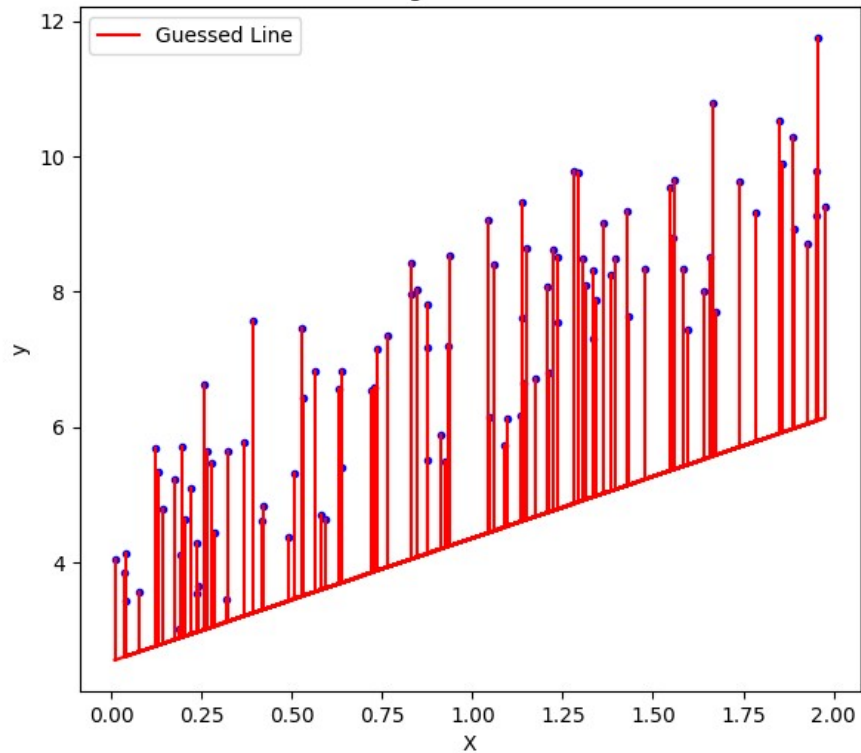


Optimizing linear regression parameters with gradient descent

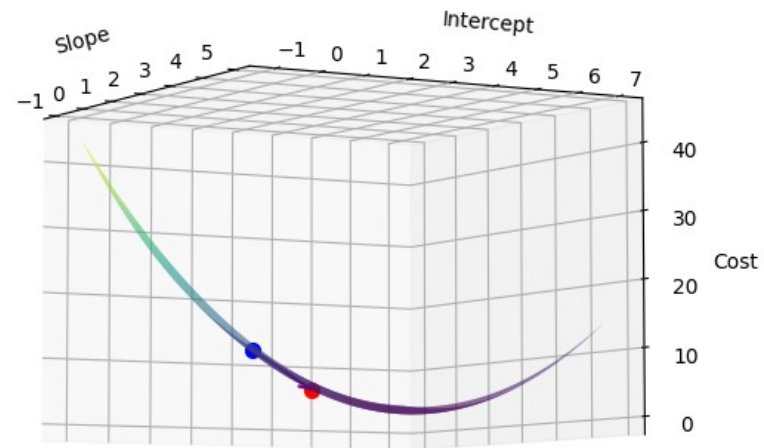
- Showing every 2 iterations out of 20 total iterations.



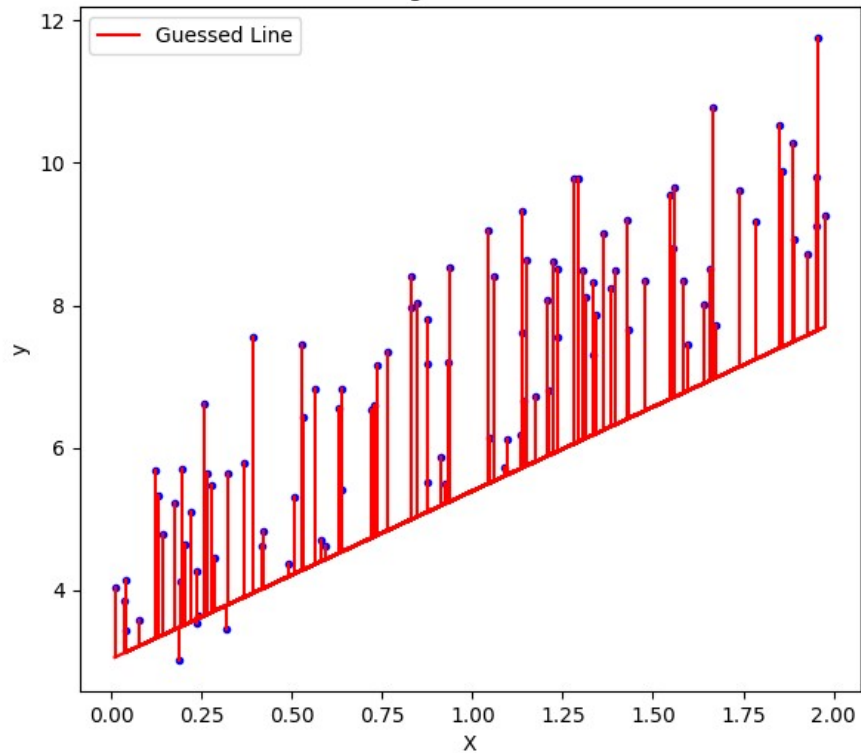
Linear Regression Iteration 4



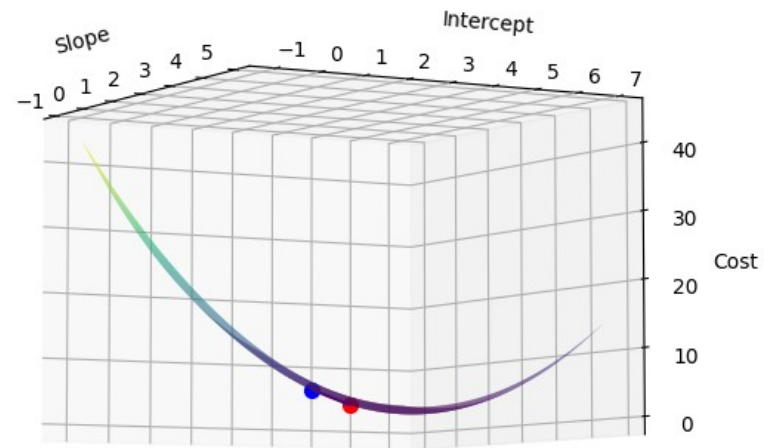
Cost Function Surface



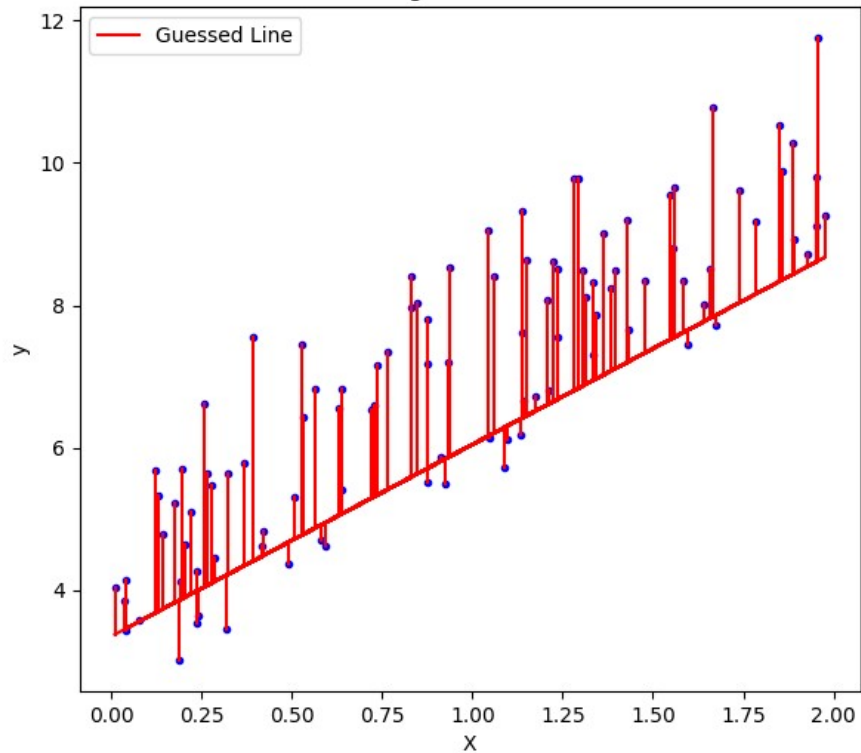
Linear Regression Iteration 6



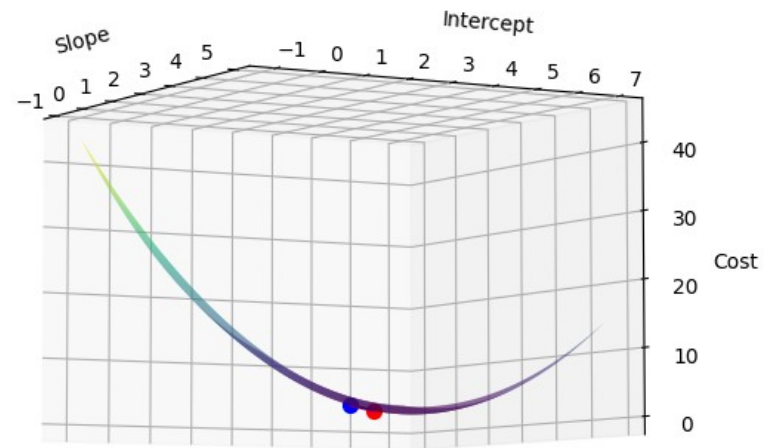
Cost Function Surface



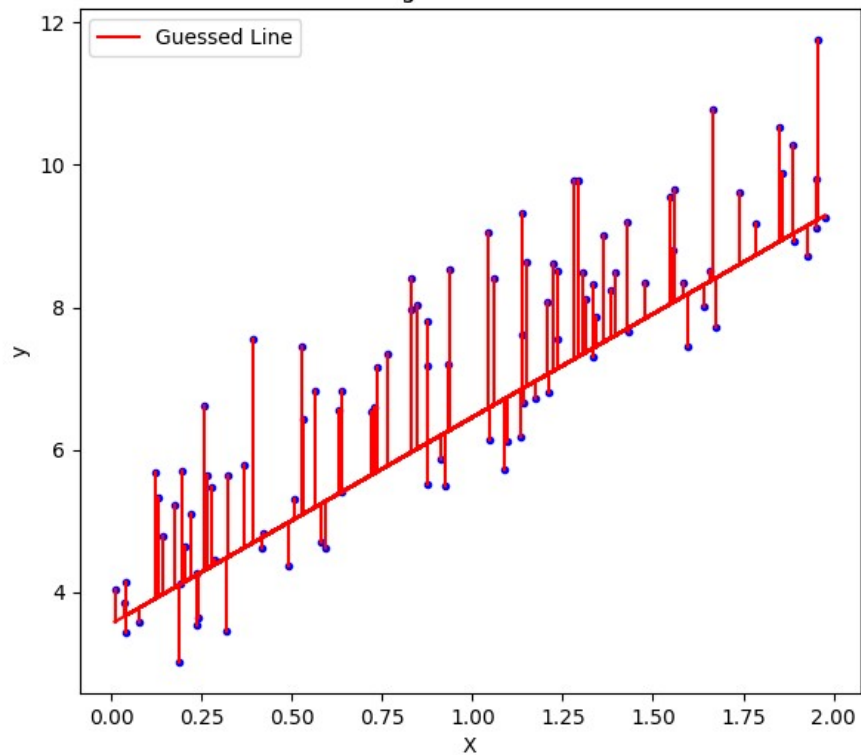
Linear Regression Iteration 8



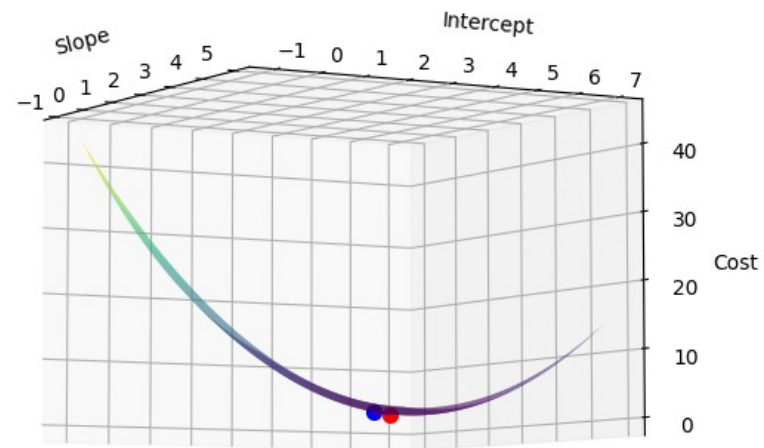
Cost Function Surface



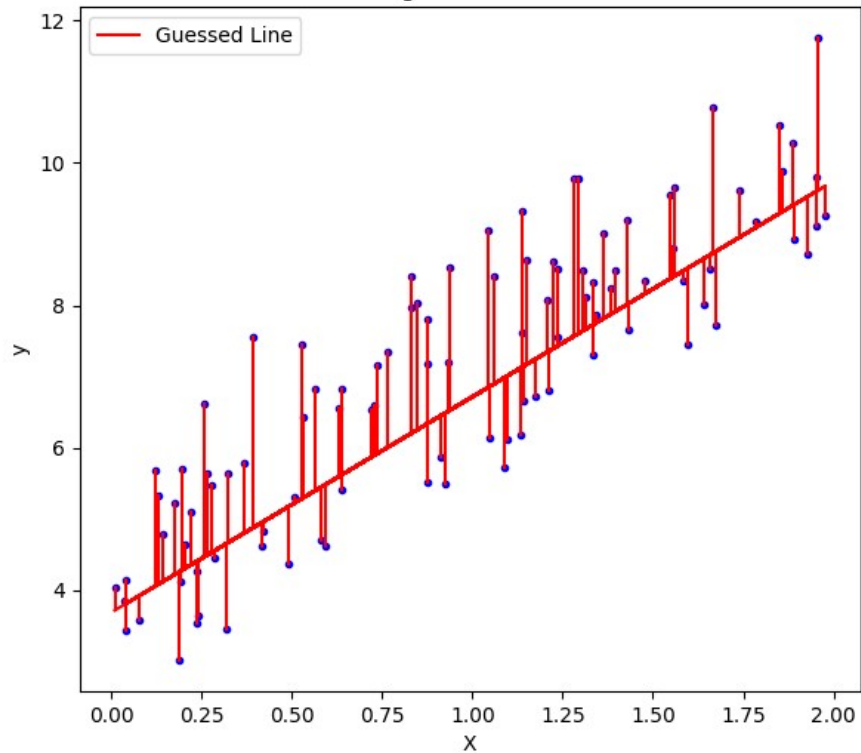
Linear Regression Iteration 10



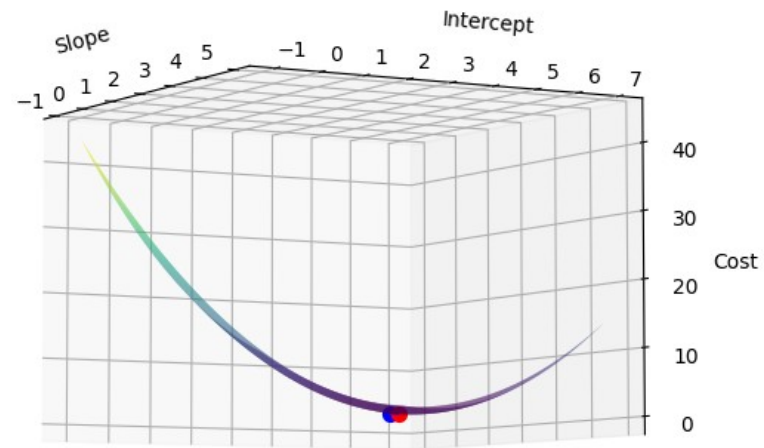
Cost Function Surface



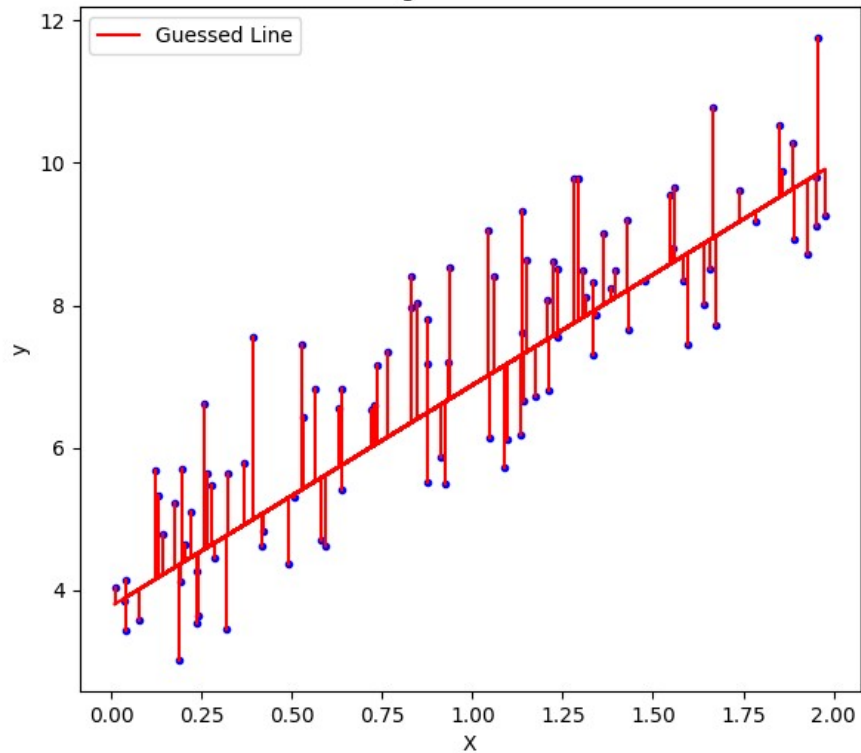
Linear Regression Iteration 12



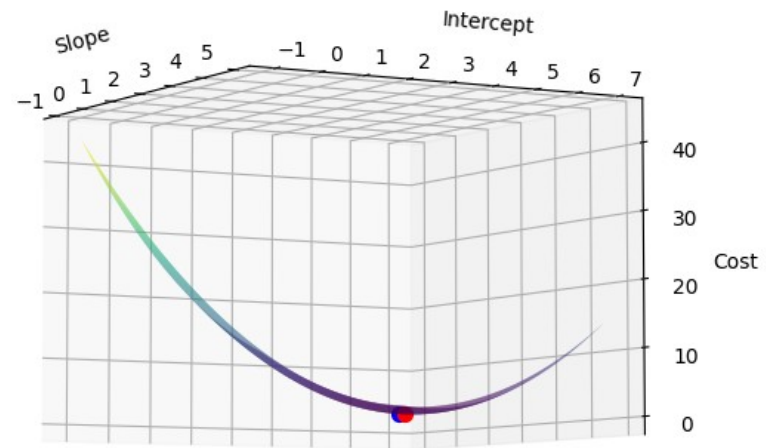
Cost Function Surface



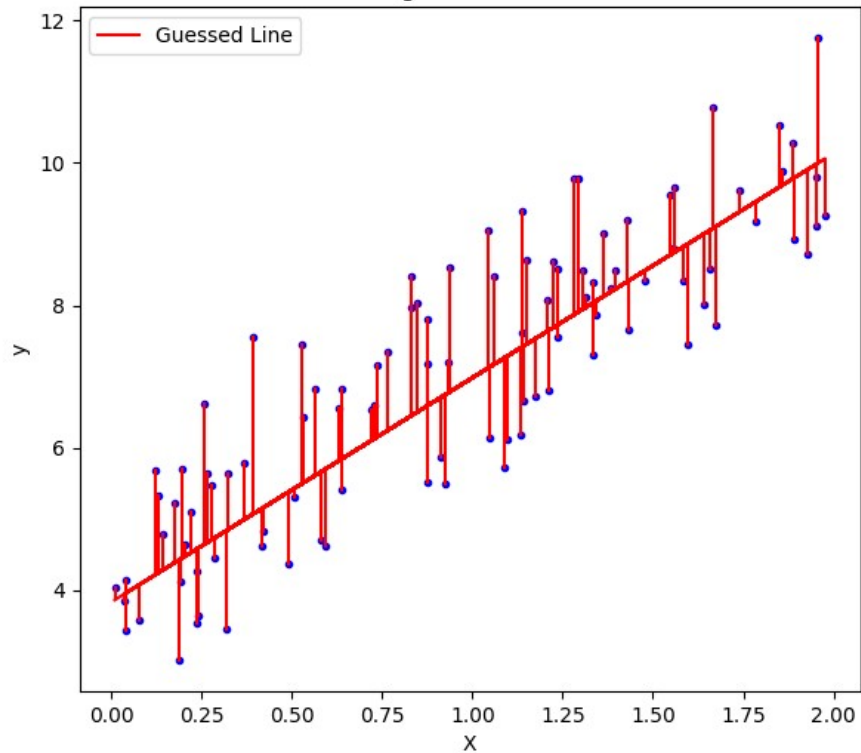
Linear Regression Iteration 14



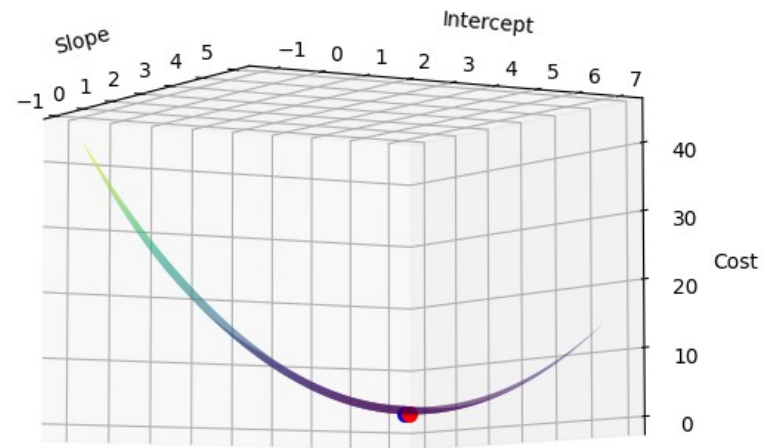
Cost Function Surface



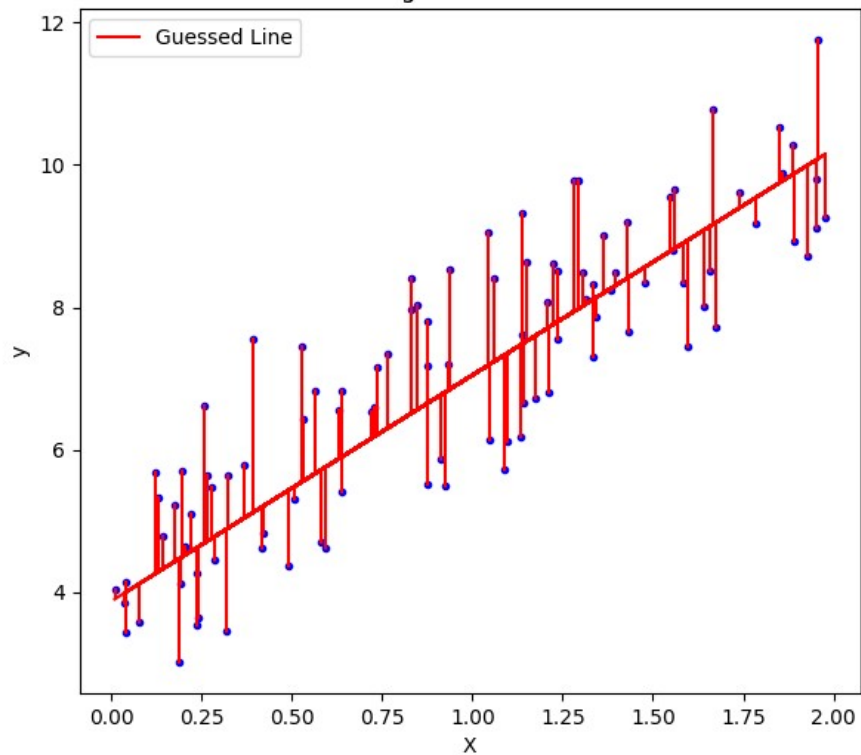
Linear Regression Iteration 16



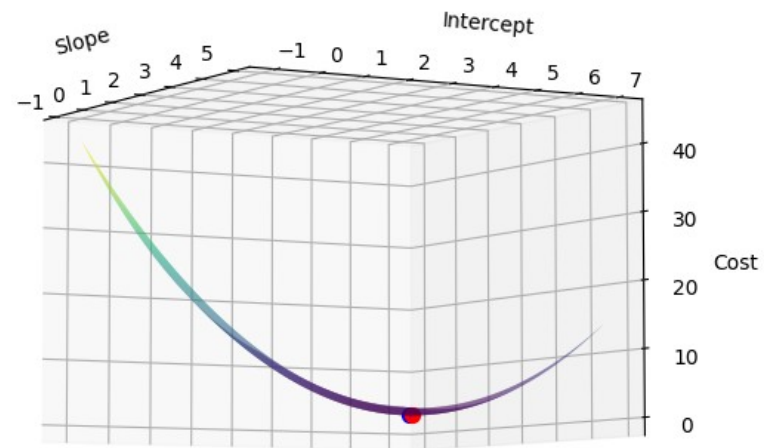
Cost Function Surface



Linear Regression Iteration 18



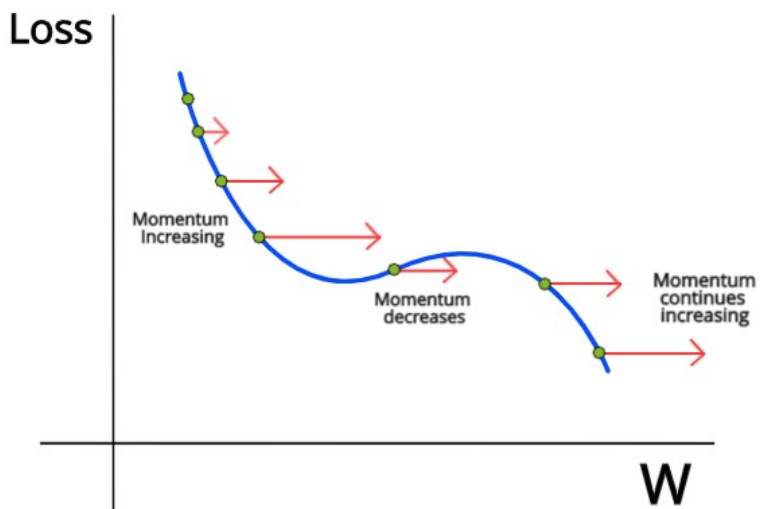
Cost Function Surface



Improving Gradient Descent

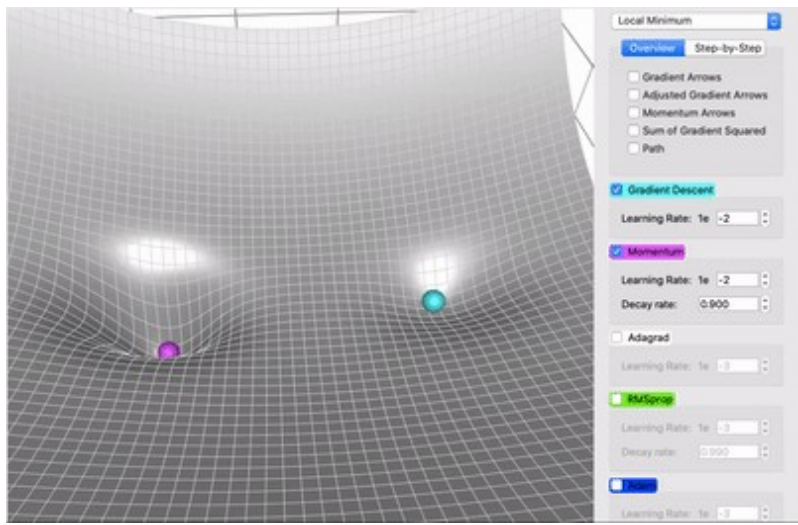
- We covered additional enhancements to simply using the directional gradient: add terms like momentum and learning rate for better control.
 - These are called **hyperparameters** of the model—not parameters of the model themselves but those used in the training or construction of the model.

Vanilla gradient descent (cyan) vs. gradient descent with momentum (purple).



Source:

<https://medium.com/deep-learning-demystified/https-medium-com-deep-learning-demystified-understanding-optimizers-312b787a60fa>



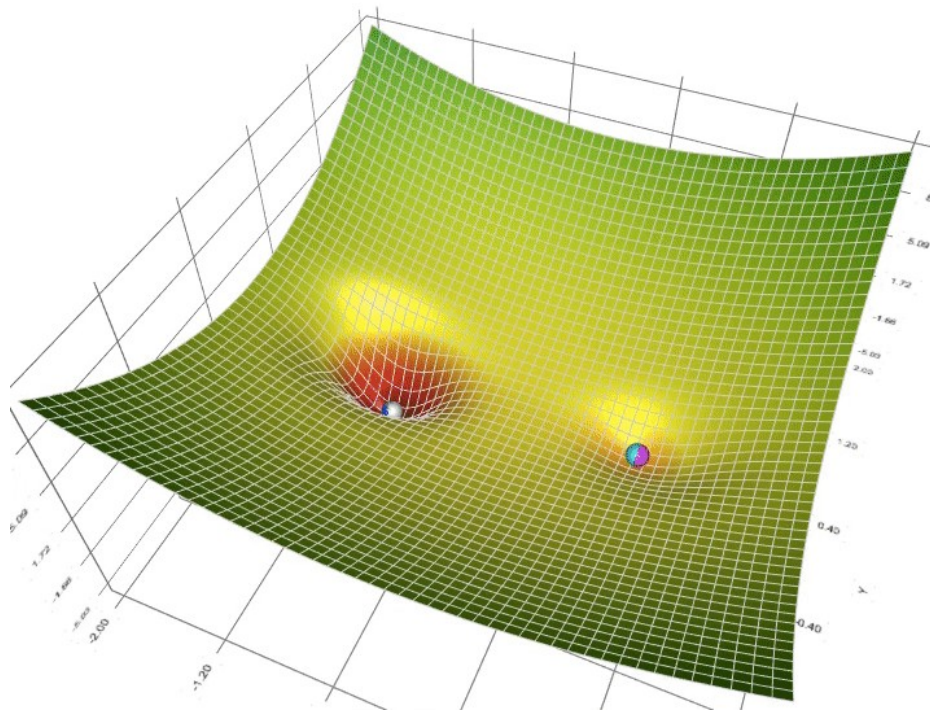
Source:

<https://towardsdatascience.com/a-visual-explanation-of-gradient-descent-methods-26c9d4a2900f>

Convex Optimization Comparison

- We discussed other variations too, such as RMSProp and Adam.
 - Visit [Lili Jiang's Towards Data Science article](#) for more examples.
 - You can also run this visualizer yourself! https://github.com/lilipads/gradient_descent_viz

Adam (blue),
RMSProp (green),
AdaGrad (white),
Momentum (purple),
Gradient Descent (cyan)



Types of Optimization

- Gradient methods work well for non-linear, differentiable problem spaces.
 - The kinds for parameters we often have in popular machine learning models.
- Optimization is a huge field!
- In management science, we often need to optimize over *linear* problems.
 - For this, we will use a technique called “linear programming”
- Note: there’s also non-linear, non-differentiable optimization methods, like genetic algorithms, that we’ll discuss later.

Example: Optimization for Decision Making

- You are a carpenter and must choose how to spend your time and resources.
- Should you build tables or bookcases?
 - Tables take 10 units of lumber and 5 hours of labor to make \$180 revenue.
 - Bookcases take 20 units of lumber and 4 hours of labor to make \$200 revenue.
 - You have 200 units of lumber and 80 hours of labor available.
- How many tables and bookcases should you build to maximize revenue?

Linear Programming

- In [linear programming](#), we express the problem using a set of linear equations:
 - The function we seek to optimize (max or min).
 - The constraints on the variables of the system, written as inequalities.

Linear Programming Example

- Let us define an axes such that **#Tables = x** and **#Bookcases = y** .
- We will express the constraints on x and y in terms of the resources.

x 

Tables take $\overbrace{10 \text{ units of lumber}}^{10x}$, $\underbrace{5 \text{ hours of labor}}_{5x}$, make \$180

y 

Bookcases take $\overbrace{20 \text{ units of lumber}}^{20y}$, $\underbrace{4 \text{ hours of labor}}_{4y}$, make \$200

200 units of lumber available

80 hours of labor available

Optimize:

$$R(x, y) = 180x + 200y$$

Constraints:

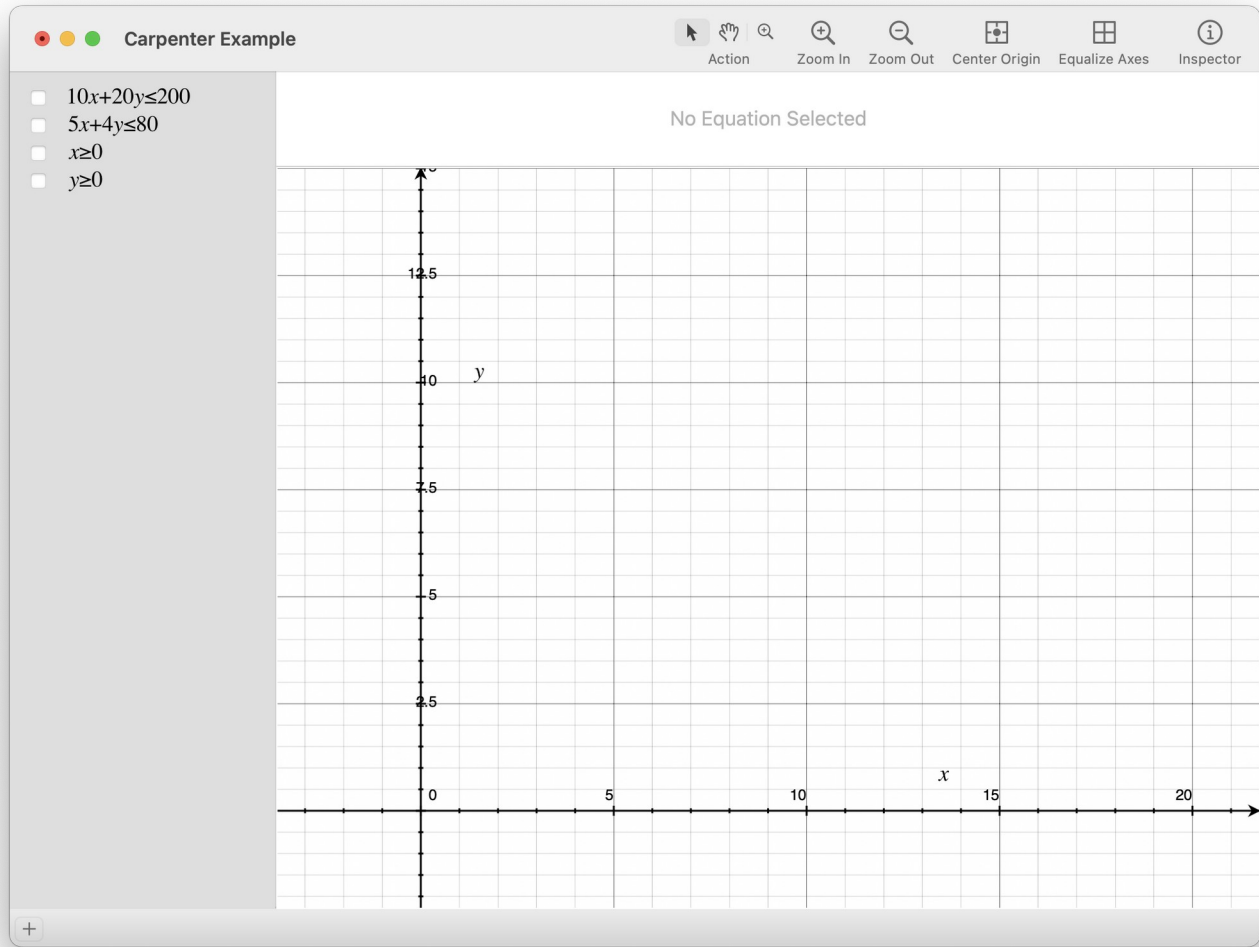
$$10x + 20y \leq 200$$

$$5x + 4y \leq 80$$

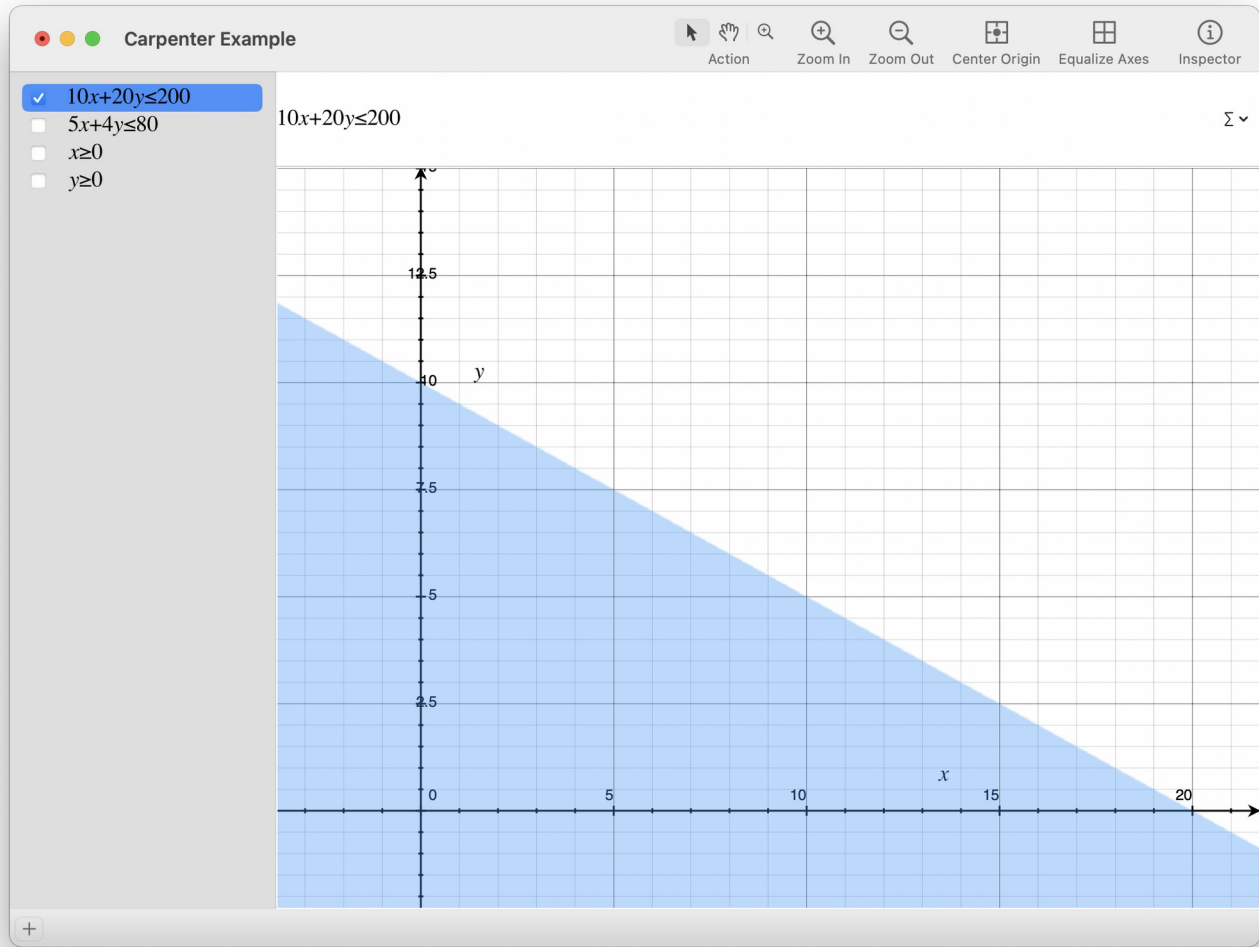
$$x \geq 0$$

$$y \geq 0$$

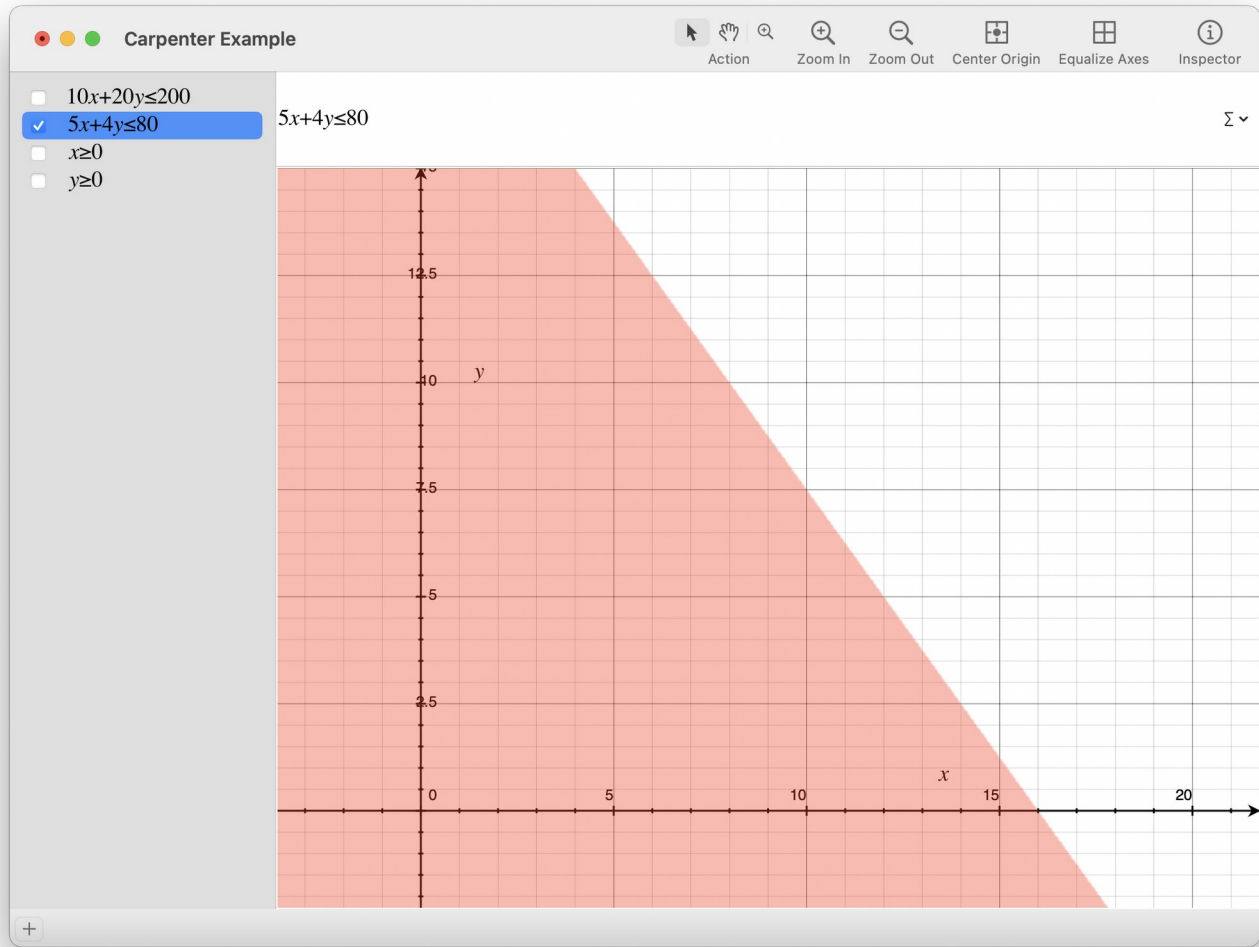
Plotting the constraints



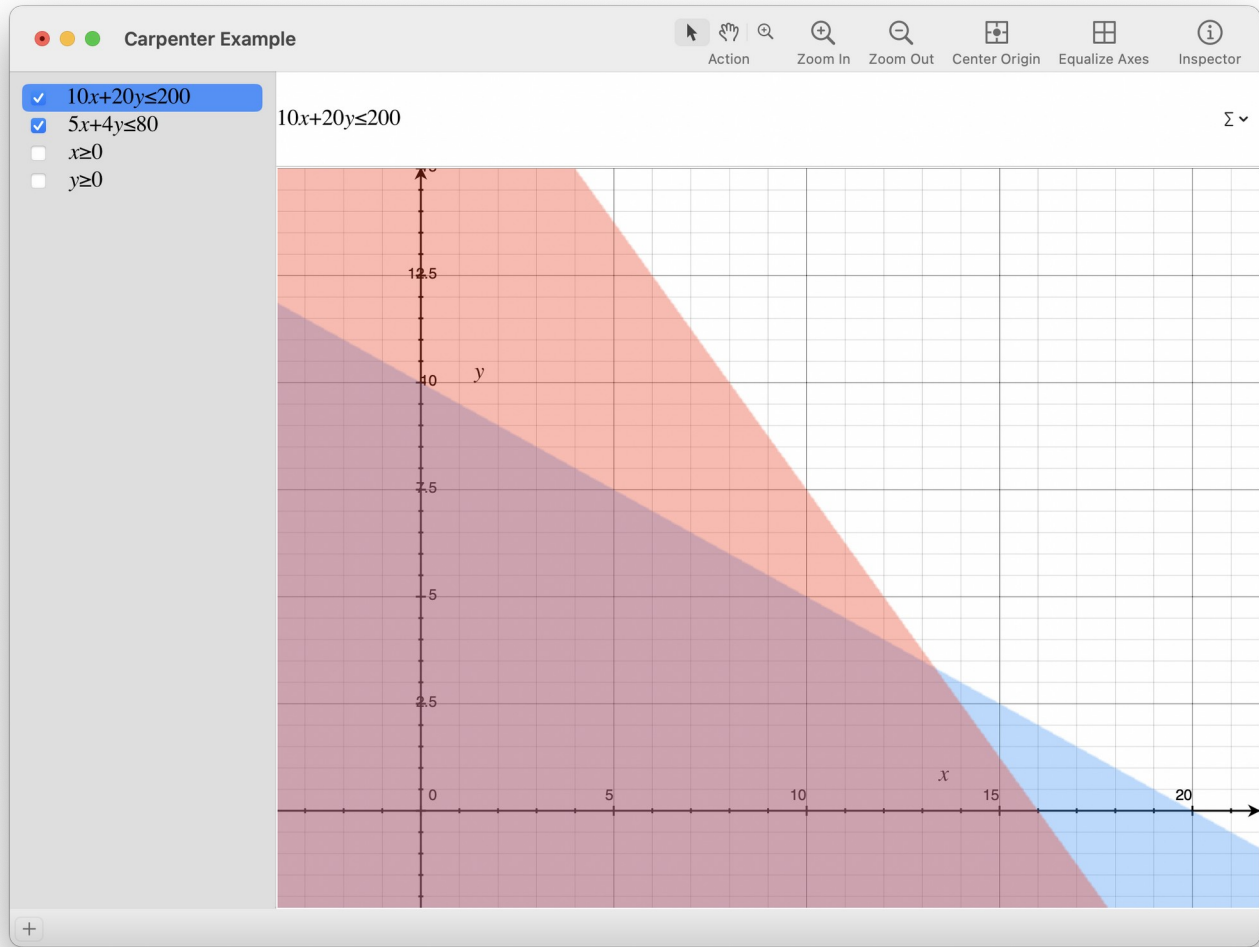
Plotting the constraints



Plotting the constraints

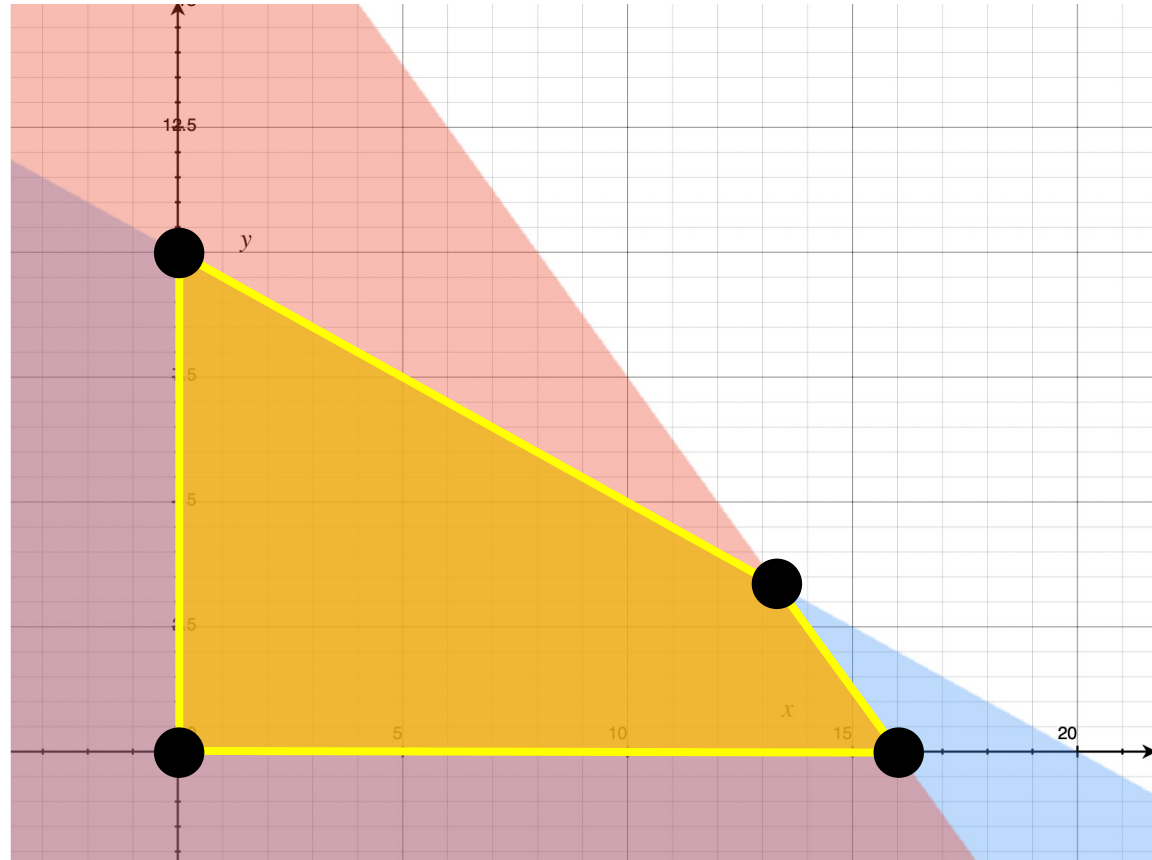


Plotting the constraints



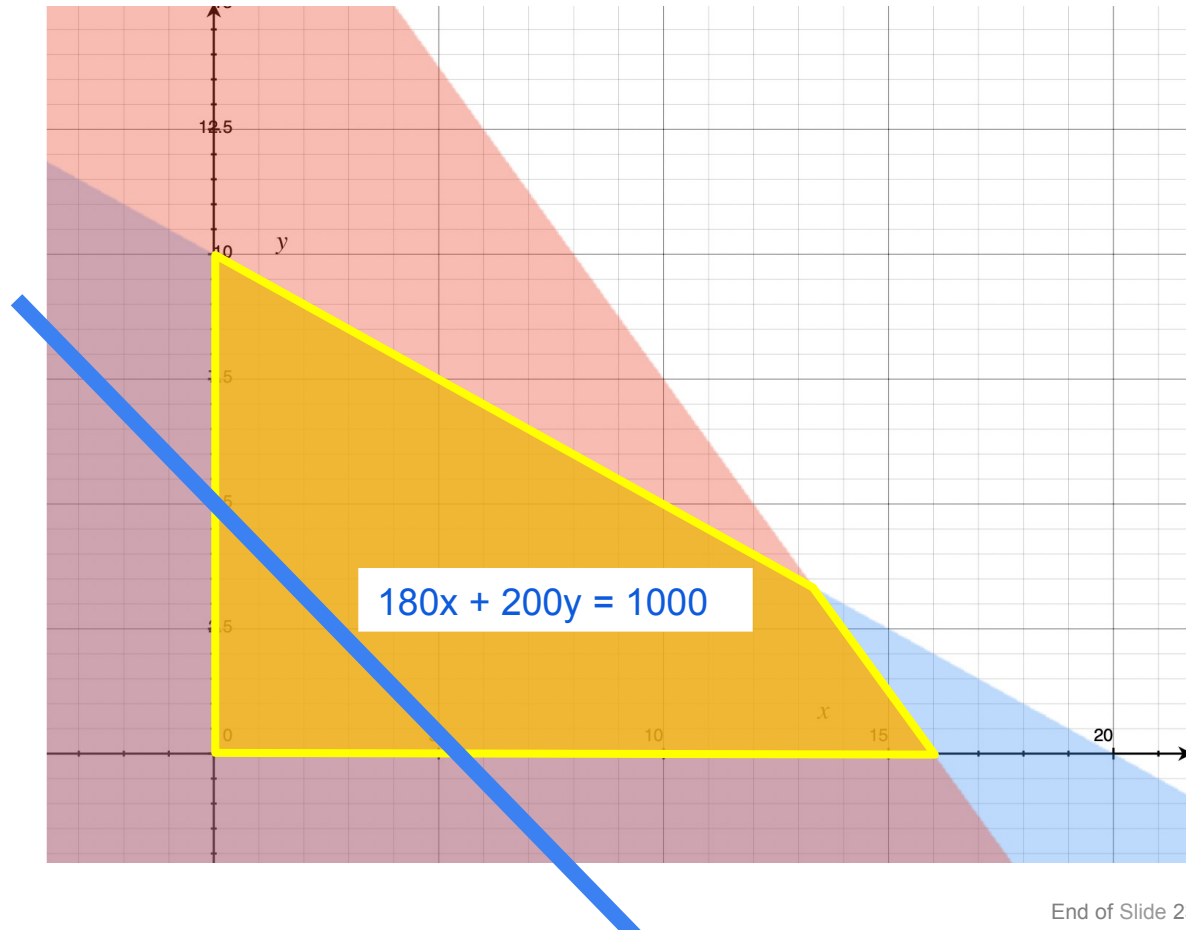
Feasible Region

- We call the overlapping area of the different inequalities, or constraints, the **feasible region**.
- This forms a convex polygon in two dimensions.
 - Generally, it will be a [convex polytype](#) across multiple dimensions.



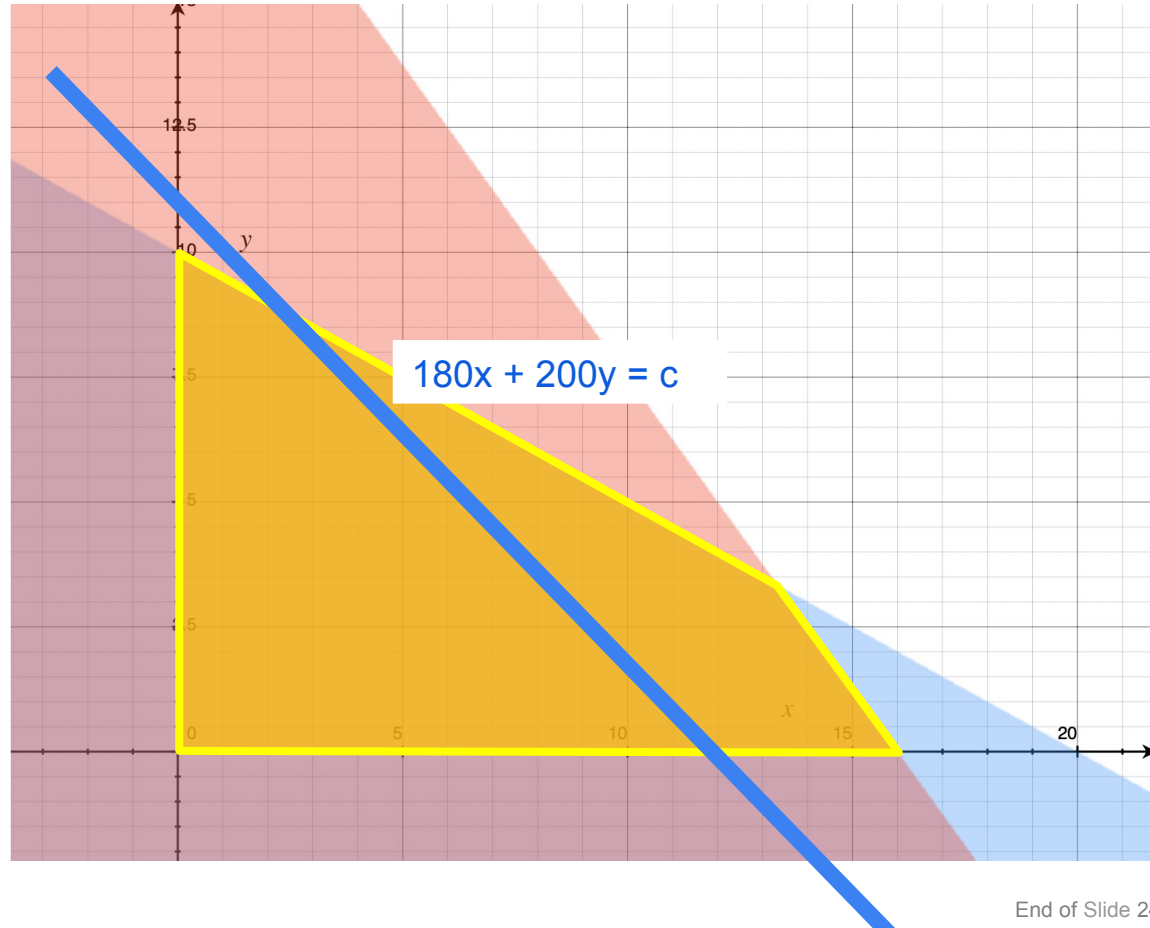
Objective Function

- Let's consider the objective function:
 - $R(x,y) = 180x + 200y$
- If we plot this for a particular value, say $R=1000$, we get the chart to the right.



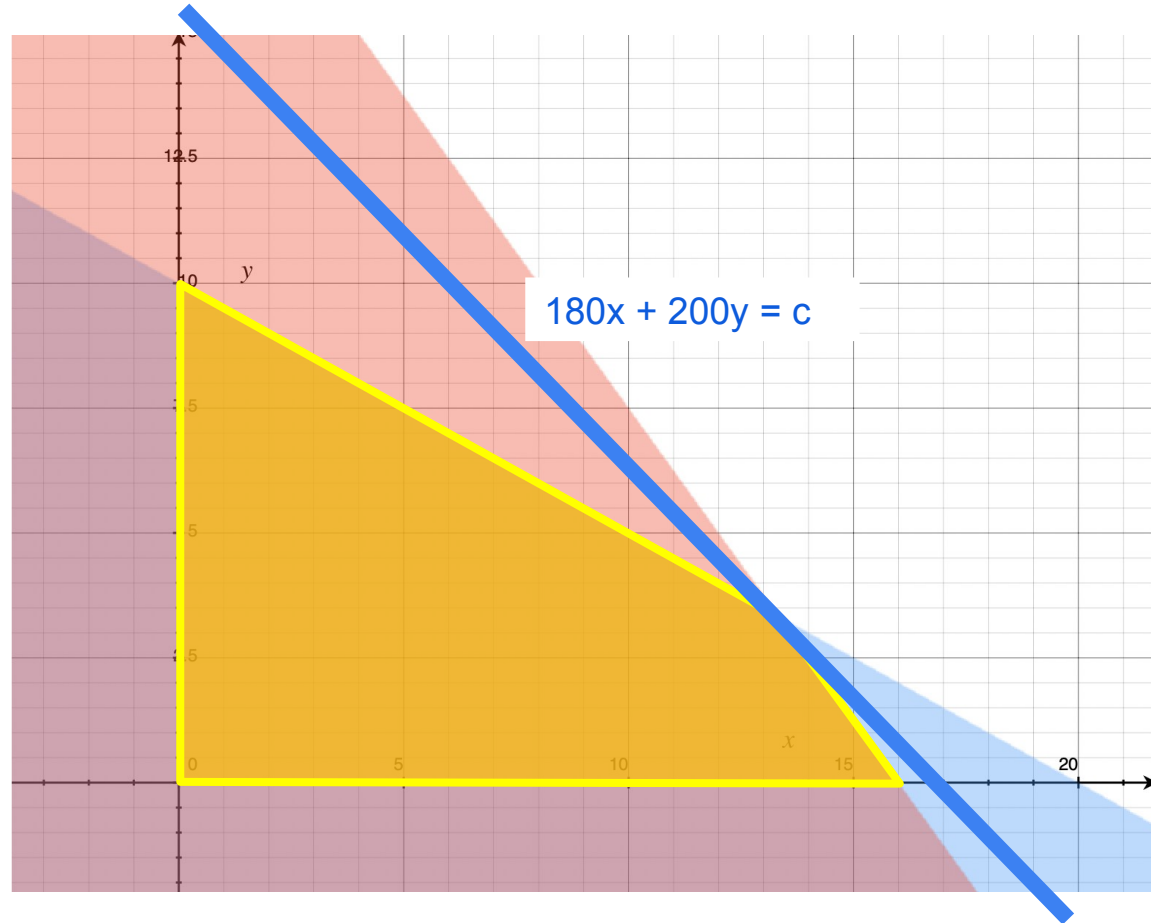
Objective Function

- Let's consider the objective function:
 - $R(x,y) = 180x + 200y$
- If we plot this for a particular value, say $R=1000$, we get the chart to the right.
- If we consider this for any constant value, c , we can visualize how the objective function overlap with the feasible region at different values.



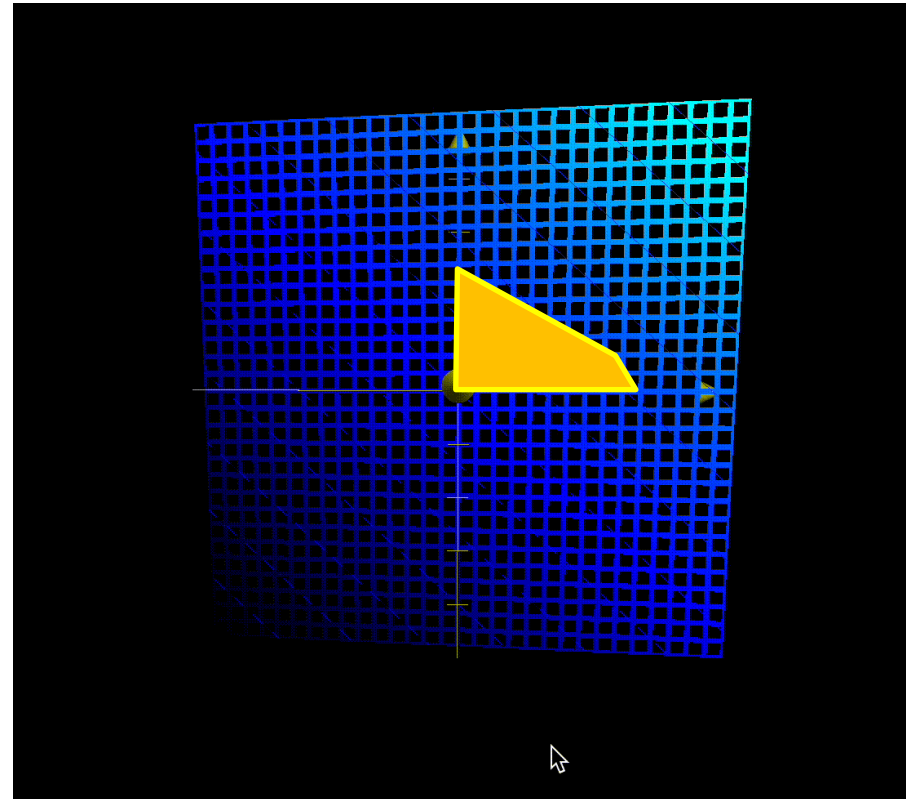
Feasible Region Vertices are Extreme Values

- When the line goes through the feasible region, there are many possible values of x and y that satisfy the equation.
- Optimal values will occur when the line intersects a single vertex.



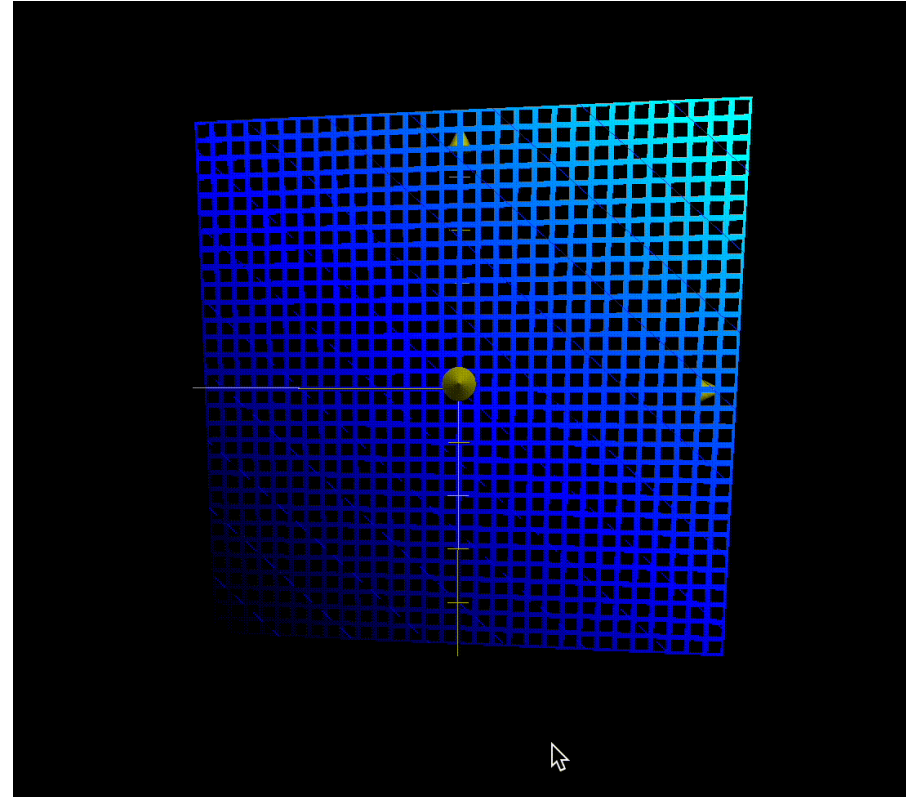
Note: Gradient Ascent Exists!

- Consider the objective function as a plane in 3D space.



Note: Gradient Ascent Exists!

- Consider the objective function as a plane in 3D space.
- We could use an optimization method like gradient ascent to move up to a maximum level of revenue.
 - Would need to incorporate additional parameters for the constraints / boundary conditions.



Linear Programming

- Rather than the calculus-based method of before, linear programming solves the problem as a system of linear equations.
 - This enables a form of optimization that is strictly algebra-based and widely used for linear optimization problems.
 - Gradient-based approaches remain a good option for non-linear optimization.

Simplex Method

- The simplex method, despite the name having “simple” in it, is a multi-step row-wise iterative process.
- Simplex optimizes the system using a technique similar to [Gaussian elimination](#) for solving a system of equations, yielding a [row echelon form matrix](#) from which the variables can be read.
- Steps:
 - 1. Re-write equations using “slack” variables to replace inequalities.
 - 2. Express as coefficient matrix.
 - 3. Select “pivot” point based on most negative column in objective row and smallest coefficient in constraint rows (using shorthand technique to simplify variable).
 - 4. Row reduce to yield column in echelon form.
 - 5. Repeat steps 3 and 4 until all system variables in reduced row echelon form.
 - 6. Assign values based on final iteration’s matrix.

Simplex Example

Example: Step 1 – Re-writing equations

- Objective should be written as equal to 0 with a positive optimization variable.
- Constraints should be written with added slack variables to remove inequalities.
- The $x, y \geq 0$ constraints are included in simplex.

Objective:

$$R = 180x + 200y$$

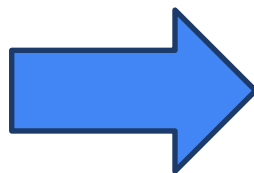
Constraints:

$$10x + 20y \leq 200$$

$$5x + 4y \leq 80$$

$$x \geq 0$$

$$y \geq 0$$



Objective:

$$-180x - 200y + R = 0$$

Constraints:

$$10x + 20y + u = 200$$

$$5x + 4y + w = 80$$

Example: Step 2 – Express as coefficient matrix or **tableau**

- This is called the "first tableau"

x	y	u	w	R	
10	20	1	0	0	200
5	4	0	1	0	80
-180	-200	0	0	1	0

Example: Step 3 – Select pivot

Pivot cell given by smallest shorthand-simplified value in pivot column

x	y	u	w	R	
10	20	1	0	0	200
5	4	0	1	0	80
-180	-200	0	0	1	0

Shorthand
simplify y to 1

200/20 = 10

80/4 = 20

Pivot column given by most negative coefficient in objective function's row

Example: Step 4 – Row reduce for current row

First, simply the row with the pivot cell.

Next, re-write all other rows so the pivot cell's coefficient (y in this case) is eliminated elsewhere.

	x	y	u	w	R	
r1	1/2	1	1/20	0	0	10
r2	5	4	0	1	0	80
r3	-180	-200	0	0	1	0

r2 − 4r1

r3 + 200r1

Example: Step 4 – Row reduce for other rows

- We are now at the second tableau

x	y	u	w	R	
1/2	1	1/20	0	0	10
3	0	-1/5	1	0	40
-80	0	10	0	1	2000

$r_2 - 4r_1$

$r_3 + 200r_1$

Example: Step 5 – Iterate steps 3 and 4 until all variables found

Pivot



x	y	u	w	R	
1/2	1	1/20	0	0	10
3	0	-1/5	1	0	40
-80	0	10	0	1	2000

Shorthand
simplify x to 1



$10 / (1/2) = 20$
 $40 / 3 = 13.33$

Pivot column given by most negative
coefficient in objective function's row



Example: Step 5 – Iterate steps 3 and 4 until all variables found

x	y	u	w	R	
1/2	1	1/20	0	0	10
1	0	-1/15	1/3	0	13.33
-80	0	10	0	1	2000

Example: Step 5 – Iterate steps 3 and 4 until all variables found

x	y	u	w	R		
0	1	1/12	-1/6	0	3.33	$r1-(1/2)r2$
1	0	-1/15	1/3	0	13.33	
0	0	14/3	80/3	1	3066.4	$r3+80r2$

Example: Step 6 – Read final values from reduced row echelon form

- We only care about the variables of our problem, not the slack variables.
- Simplex yielded the following solution:
 - $X = 13.33$
 - $Y = 3.33$
 - $R = 3066.4$

x	y	R	u	w	
1	0	0	-1/15	1/3	13.33
0	1	0	1/12	-1/6	3.33
0	0	1	14/3	80/3	3066.4

Let's do this in Excel!

Setting up the spreadsheet

- The exact arrangement doesn't matter.
- You need:
 - Empty cells for the variables whose values will be changed.
 - Formula cells containing your equations.

	A	B	C	D	E	F
1		X	Y			
2						
3	Max Revenue	180	200	$=B3*B2+C3*C2$		
4	Constraint 1	10	20	0	<=	200
5	Constraint 2	5	4	0	<=	80

Running the Solver

- Set all the parameters in the solver dialog for Simplex:

D3

fx

=B3*B2+C3*C2

	A	B	C	D	E	F
1		X	Y			
2						
3	Max Revenue	180	200	0		
4	Constraint 1	10	20	0	<=	200
5	Constraint 2	5	4	0	<=	80
6						
7						
8						
9						
10						
11						
12						
13						
14						
15						
16						

Solver Parameters

Set Objective:

\$D\$3

To:

☒ Max

☐ Min

☐ Value Of:

0

By Changing Variable Cells:

\$B\$2:\$C\$2

Subject to the Constraints:

\$D\$4 <= \$F\$4

\$D\$5 <= \$F\$5

Add

Change

Delete

Reset All

Load/Save

☒ Make Unconstrained Variables Non-Negative

Select a Solving Method:

Simplex LP

Options

Solving Method

Select the GRG Nonlinear engine for Solver Problems that are smooth nonlinear. Select the LP Simplex engine for linear Solver Problems, and select the Evolutionary engine for Solver problems that are non-smooth.

Close

Solve

We get the same result!

	A	B	C	D	E	F
1		X	Y			
2		13.33333333	3.333333333			
3	Max Revenue	180	200	3066.66667		
4	Constraint 1	10	20	200 <=		200
5	Constraint 2	5	4	80 <=		80

Note 1: You can save reports from the solver's output.

- This captures interesting measures, like the “shadow price” which is how much the constraints (lumber and labor in our example) contributed to the revenue within certain bounds.
- For more information about Excel’s linear programming reports, see [Graduate Tutor’s Reference Site](#)

6	Variable Cells						
7			Final	Reduced	Objective	Allowable	Allowable
8	Cell	Name	Value	Cost	Coefficient	Increase	Decrease
9	\$B\$2 X		13.33333333	0	180	70	80
10	\$C\$2 Y		3.333333333	0	200	160	56
11							
12	Constraints						
13			Final	Shadow	Constraint	Allowable	Allowable
14	Cell	Name	Value	Price	R.H. Side	Increase	Decrease
15	\$D\$4	Constraint 1	200	4.666666667	200	200	40
16	\$D\$5	Constraint 2	80	26.66666667	80	20	40
17							
18							
19							

◀ ▶

Answer Report 1

Sensitivity Report 1

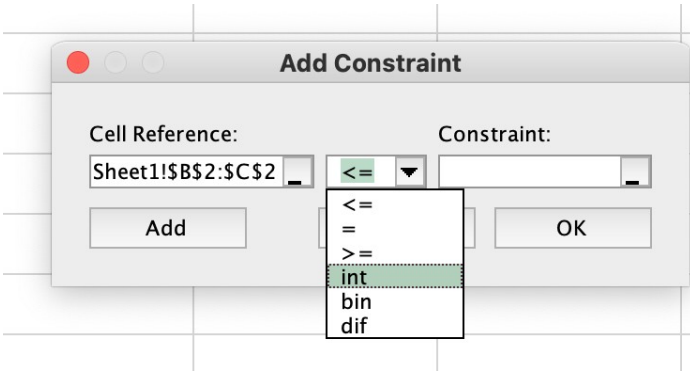
Limits Report 1

Sheet1

+

Note 2: Applying Integer Constraints

- When working with goods, we need to optimize to whole numbers.
 - Rounding is one quick fix.
- Excel can apply an integer constraint.
 - Ensure that “Ignore Integer Constraints” is not checked under “Options” to use this feature.
- With integer constraints, we get the solution of 12 tables and 4 bookshelves:



	X	Y			
	12	4			
Max Revenue	180	200	2960		
Constraint 1	10	20	200	<=	200
Constraint 2	5	4	76	<=	80

Gradescope Practice

Supply Chain Management Example

- Let us consider the problem faced by Foster Manufacturing, Inc., which produces commercial generators.
- Currently Foster has three plants: one in Cleveland, Ohio; one in Bedford, Indiana; and one in York, Pennsylvania.
- Generators produced at the plants (supply points) are shipped to distribution centers (demand locations) in Boston, Chicago, St. Louis, and in Lexington, Kentucky.
- The question is how to best allocate production among the distribution centers at minimum cost.

Credit: <https://www.youtube.com/watch?v=EkrbhiCEvvQ>

We seek to minimize cost under supply and demand constraints.

	A	B	C	D	E	F
1	Foster Manufacturing					
2						
3	Data	Boston	Chicago	St. Louis	Lexington	Supply
4	Cleveland	\$3.00	\$2.00	\$7.00	\$6.00	5000
5	Bedford	\$7.00	\$5.00	\$2.00	\$3.00	6000
6	York	\$2.00	\$5.00	\$4.00	\$5.00	2500
7	Demand	6000	4000	2000	1500	
8						
9	Model					
10						
11	Decision variables	Boston	Chicago	St. Louis	Lexington	
12	Cleveland					
13	Bedford					
14	York					
15						
16	Supply Constraints	Cleveland	Bedford	York		
17	Amount Shipped From	0	0	0		
18						
19	Demand Constraints	Boston	Chicago	St. Louis	Lexington	
20	Amount Shipped To	0	0	0	0	
21						
22		Cost				
23	Total	\$ -				

Result of solver:

	A	B	C	D	E	F
1	Foster Manufacturing					
2						
3	Data	Boston	Chicago	St. Louis	Lexington	Supply
4	Cleveland	\$3.00	\$2.00	\$7.00	\$6.00	5000
5	Bedford	\$7.00	\$5.00	\$2.00	\$3.00	6000
6	York	\$2.00	\$5.00	\$4.00	\$5.00	2500
7	Demand	6000	4000	2000	1500	
8						
9	Model					
10						
11	Decision variables	Boston	Chicago	St. Louis	Lexington	
12	Cleveland	3500	1500	0	0	
13	Bedford	0	2500	2000	1500	
14	York	2500	0	0	0	
15						
16	Supply Constraints	Cleveland	Bedford	York		
17	Amount Shipped From	5000	6000	2500		
18						
19	Demand Constraints	Boston	Chicago	St. Louis	Lexington	
20	Amount Shipped To	6000	4000	2000	1500	
21						
22		Cost				
23	Total	\$ 39,500.00				

Excel's Solver has other algorithms as well...

- We used Simplex LP for “linear programming” on the linear optimization problems.
- Excel also has a GRG-nonlinear model.
 - GRG stands for “Generalized Reduced Gradient” and is, as you might expect from the name, a gradient-based optimization method for non-linear objective functions.
 - E.g., a neural network’s cost or error function (thanks largely to the activation part of the model)
- Those methods both expect the objective function to be *smooth*.
 - This essentially means it is differentiable over the feasible region.
- For non-linear and non-smooth objective functions, we need different optimization methods.
 - Evolutionary algorithms are generally applied in this space, using what we call mutations as the trial-and-error step for searching for optimal values.

References

- Much of this lecture content was adapted from various online resources, and you are encouraged to visit them for additional details:
 - Carpenter problem: <https://www.youtube.com/watch?v=K7TL5NMIKIk>
 - Simplex example: https://www.youtube.com/watch?v=rzRZLGD_aeE
 - Excel examples: <https://www.youtube.com/watch?v=32JaLADHL8c> and https://www.youtube.com/watch?v=vi0bCm_AQv0
 - Supply chain example: <https://www.youtube.com/watch?v=EkrbhiCEvvQ>